

## *Employee Table Schema with Data Integrity Rules*

**Created By: Sharu latha. B**

**Course: AI-Driven Data Analytics**

**Batch No: FNB16**

### **Project Objectives**

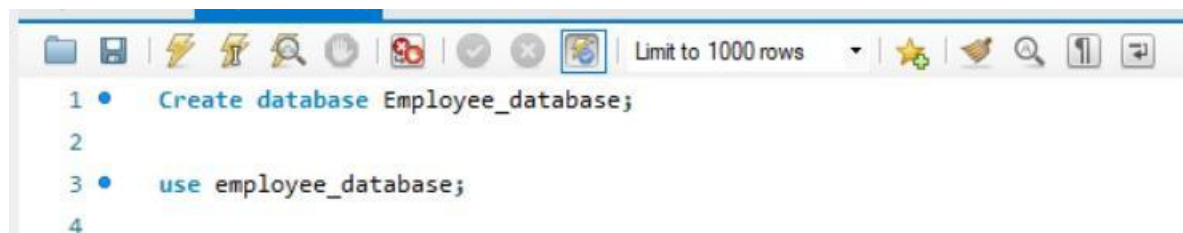
The objective of this project is to design and implement a relational database system for managing employee information. The project focuses on applying SQL constraints to ensure data integrity, consistency, and reliability across multiple tables.

### **MySQL DDL Commands**

- Create
- Alter
- Drop
- Truncate
- Rename

### **Creating Employee Database with Constraints**

#### **Database and Table Creation (CREATE):**



```
1 • Create database Employee_database;  
2  
3 • use employee_database;  
4
```

#### **Importance of Creating Tables Inside a Database**

##### **Organized Data Storage**

- ✦ Tables provide a structured way to store information in rows and columns.
- ✦ Each table represents a specific entity (e.g. Employees, Departments, Locations).

##### **Data Integrity**

- ✦ Constraints (PRIMARY KEY, FOREIGN KEY, NOT NULL, CHECK) ensure that only valid data is stored.
- ✦ Prevents duplication and enforces rules automatically.

## Table Creation (CREATE):

Example:

|                               |
|-------------------------------|
| Table: Departments            |
| Attributes:                   |
| • Department id – Primary Key |
| • department name             |

```

4
5  --- Create department table-
6
7  • Create table Departments(
8    department_id int auto_increment primary key,
9    department_name varchar(50) not null unique
10 );
11
12 • select * from departments;

```

Result Grid | Filter Rows: | Edit: | Export/Import: | Wrap Cell Content: [IA](#)

| department_id | department_name |
|---------------|-----------------|
| NULL          | NULL            |

|                             |
|-----------------------------|
| Table: Location             |
| Attributes:                 |
| ✦ location id – Primary Key |
| ✦ location name             |

```

13
14   --- create table Location-
15
16 • Create table Location(
17   location_id int auto_increment primary key,
18   location_name varchar(100) not null unique
19   );
20
21 • desc Location;

```

Result Grid | Filter Rows: | Export: | Wrap Cell Content: [IA](#)

|   | Field         | Type         | Null | Key | Default     | Extra          |
|---|---------------|--------------|------|-----|-------------|----------------|
| ▶ | location_id   | int          | NO   | PRI | <b>NULL</b> | auto_increment |
|   | location_name | varchar(100) | NO   | UNI | <b>NULL</b> |                |

Table: Employees

Attributes:

- ✦ employee id – Primary Key
- ✦ Employee name
- ✦ Gender
- ✦ Age
- ✦ Hire date
- ✦ Designation
- ✦ Salary
- ✦ Department id – Foreign Key referencing department (department id)
- ✦ location id—Foreign Key referencing location (location id)

**SCHEMAS**

Filter objects

- analysis\_db
  - employee\_database
    - Tables
      - departments
      - employees
      - location
    - Views
    - Stored Procedures
    - Functions
  - hospitaldb
  - sakila
  - socialmediadb
  - sys
  - world

Administration Schemas

Information

**Table: employees**

**Columns:**

|               |               |
|---------------|---------------|
| employee_id   | int AI PK     |
| department_id | int           |
| location_id   | int           |
| employee_name | varchar(100)  |
| gender        | enum('M','F') |
| age           | int           |
| hire_date     | date          |
| designation   | varchar(100)  |
| salary        | int           |

```

25 • create table employees(
26     employee_id int auto_increment primary key,
27     department_id int,
28     location_id int,
29     employee_name varchar(100) not null,
30     gender enum('M','F'),
31     age int check (age >= 18),
32     hire_date date default (current_date),
33     designation varchar(100),
34     salary int,
35     foreign key (department_id) references departments (department_id),
36     foreign key(location_id) references location (location_id)
37 );
38
39 • select * from employees;
  
```

Result Grid

|   | employee_id | department_id | location_id | employee_name | gender | age  | hire_date | designation | salary |
|---|-------------|---------------|-------------|---------------|--------|------|-----------|-------------|--------|
| * | NULL        | NULL          | NULL        | NULL          | NULL   | NULL | NULL      | NULL        | NULL   |

employees 3 x

## Table Alteration (ALTER)

- ✦ Add a new column named "email" to the Employees table to store employee email addresses.

```

39 • select * from employees;
40 • use employee_database;
41
42 --- Alter table-Add email column-
43 • Alter table employees
44     add column Email varchar(100) unique;
45
46 • select * from employees;
  
```

Result Grid

|   | employee_id | department_id | location_id | employee_name | gender | age  | hire_date | designation | salary | Email |
|---|-------------|---------------|-------------|---------------|--------|------|-----------|-------------|--------|-------|
| * | NULL        | NULL          | NULL        | NULL          | NULL   | NULL | NULL      | NULL        | NULL   | NULL  |

- ✦ Modify the data type of the "designation" column in the Employees table to support a wider range of values.

```

48      --- Modify the data type of the destination column- wider range of value--
49
50 •    Alter table employees
51      Modify designation varchar(400);
52

```

✦ Drop the “age” column from the Employees table.

```

54
55      --- drop age column from employee table--
56
57 •    alter table employees
58      drop column age;
59
60

```

Table: **employees**

Columns:

|                      |               |
|----------------------|---------------|
| <b>employee_id</b>   | int AI PK     |
| <b>department_id</b> | int           |
| <b>location_id</b>   | int           |
| employee_name        | varchar(100)  |
| gender               | enum('M','F') |
| hire_date            | date          |
| designation          | varchar(400)  |
| salary               | int           |

## Table Renaming (RENAME):

- ✦ Rename the “hire date” column to “date of joining”.

```

61
62 •   Alter table employees
63     rename column hire_date to date_of_joining;
64
65
66
--

```

Table: **employees**

Columns:

|                      |               |
|----------------------|---------------|
| <b>employee_id</b>   | int AI PK     |
| <b>department_id</b> | int           |
| <b>location_id</b>   | int           |
| employee_name        | varchar(100)  |
| gender               | enum('M','F') |
| date_of_joining      | date          |
| designation          | varchar(400)  |
| salary               | int           |

- ✦ Rename the "Departments" table to "Departments Info".
- ✦ Rename the "Location" table to "Locations".

```

--
55   --- Table Rename departments to department_info--
56
57 •   Rename table departments to department_info;
58
59 •   Rename table location to locations;
60
61
62
--

```

## Table Truncation (TRUNCATE):

- ✦ Truncate the Employees table

```

71    --- Truncate employees table --
72
73 •   truncate table employees;
74

```

## Database & Table Dropping (DROP):

```

78
79    /* Database and table dropping */
80
81 •   drop table employees;
82 •   drop database employee_database;
83
84

```

## Database Recreation:

- ✦ Recreate it using the provided schema, ensuring that all tables are created with the appropriate constraints as instructed.

```

/* database recreation */

•   drop database employee_database;

•   Create database Employee_database;

•   use employee_database;

```

- ✦ Establish links between the "department id" and "location id" fields in the "employees" table and their respective tables.

```
--- create table employees-
```

```
create table employees(
  employee_id int auto_increment primary key,
  department_id int,
  location_id int,
  employee_name varchar(100) not null,
  gender enum('M','F'),
  age int check (age >= 18),
  hire_date date default (current_date),
  designation varchar(100),
  salary int,
  foreign key (department_id) references departments (department_id),
  foreign key(location_id) references location (location_id)
);
```

## Departments Table:

- ✦ Ensure that the "department id" uniquely identifies each department.

```
/* Ensure that the "department_id" uniquely identifies each department */
```

```
insert into departments(department_name)
values('HR'),('Finance'),('IT'),('Sales');
```



```

125      /* Ensure that the "department_id" uniquely identifies each department */
126
127 •    insert into departments(department_name)
128      values('HR'),('Finance'),('IT'),('Sales');
129
130 •    select * from departments;

```

| Result Grid |               | Filter Rows:    | Edit: | Export/Import: | Wrap Cell Content: |
|-------------|---------------|-----------------|-------|----------------|--------------------|
|             | department_id | department_name |       |                |                    |
| ▶           | 1             | HR              |       |                |                    |
|             | 2             | Finance         |       |                |                    |
|             | 3             | IT              |       |                |                    |
|             | 4             | Sales           |       |                |                    |
| •           | NULL          | NULL            |       |                |                    |

```

• Create table Departments(
    department_id int auto_increment primary key,
    department_name varchar(50) not null unique
);

```

```

ALTER TABLE departments
ADD CONSTRAINT UNIQUE (department_id);

```

✦ Set up constraints on the "department\_name" to avoid duplicate and null entries.

```

132      /* Set up constraints on the "department_name" to avoid duplicate and null entries */
133
134
135      • SELECT *
136      FROM departments
137      WHERE department_name is not null;
138

```

| Result Grid |               | Filter Rows:    | Edit: | Export/Import: | Wrap Cell Content: |
|-------------|---------------|-----------------|-------|----------------|--------------------|
|             | department_id | department_name |       |                |                    |
| ▶           | 1             | HR              |       |                |                    |
|             | 2             | Finance         |       |                |                    |
|             | 3             | IT              |       |                |                    |
|             | 4             | Sales           |       |                |                    |
| •           | NULL          | NULL            |       |                |                    |

```

Create table Location(
  location_id int auto_increment primary key,
  location_name varchar(100) not null unique
);

```

## Locations Table:

- ✦ Establish a mechanism to automatically generate unique identifiers for each location, ensuring that they are incremented sequentially.

```

139      /* Locations Table
140      • Establish a mechanism to automatically generate unique identifiers for each
141      location, ensuring that they are incremented sequentially */
142
143      • insert into location( location_name)
144      values('Chennai'),('Coimbatore'),('Mumbai'),('Bangalore');
145
146      • select * from location;
147

```

| Result Grid |             | Filter Rows:  | Edit: | Export/Import: | Wrap Cell Content: |
|-------------|-------------|---------------|-------|----------------|--------------------|
|             | location_id | location_name |       |                |                    |
| ▶           | 1           | Chennai       |       |                |                    |
|             | 2           | Coimbatore    |       |                |                    |
|             | 3           | Mumbai        |       |                |                    |
|             | 4           | Bangalore     |       |                |                    |
| •           | NULL        | NULL          |       |                |                    |

- ✦ Implement constraints to prevent the insertion of null and duplicate locations.

```

148      /* Implement constraints to prevent the insertion of null and duplicate locations */
149
150      SELECT *
151      FROM location
152      WHERE location_name is not null;
153

```

Result Grid | Filter Rows: | Edit: | Export/Import: | Wrap Cell Content: |

|   | location_id | location_name |
|---|-------------|---------------|
| ▶ | 1           | Chennai       |
|   | 2           | Coimbatore    |
|   | 3           | Mumbai        |
|   | 4           | Bangalore     |
| • | NULL        | NULL          |

## Employees Table:

- ✦ Guarantee that each employee has a distinct identifier.

```

158      /* Employees Table:
159      Guarantee that each employee has a distinct identifier */
160
161      INSERT INTO employees (employee_name, gender, age, hire_date, designation, salary, department_id, location_id)
162      VALUES ('Arun Kumar', 'M', 28, '2025-01-01', 'developer', 45000, 1, 1),
163      ('Karthik', 'M', 35, '2025-01-20', 'Finance Officer', 55000, 2, 2);
164
165      select * from employees;

```

Result Grid | Filter Rows: | Edit: | Export/Import: | Wrap Cell Content: |

|   | employee_id | department_id | location_id | employee_name | gender | age  | hire_date  | designation     | salary |
|---|-------------|---------------|-------------|---------------|--------|------|------------|-----------------|--------|
| ▶ | 1           | 1             | 1           | Arun Kumar    | M      | 28   | 2025-01-01 | developer       | 45000  |
|   | 2           | 2             | 2           | Karthik       | M      | 35   | 2025-01-20 | Finance Officer | 55000  |
| • | NULL        | NULL          | NULL        | NULL          | NULL   | NULL | NULL       | NULL            | NULL   |

- ✦ Create a restriction to ensure that the employee's name is always provided.

```

167      /* Create a restriction to ensure that the employee's name is always provided */
168
169      select * from employees
170      WHERE employee_name is not null;
171

```

| Result Grid   Filter Rows:   Edit:   Export/Import:   Wrap Cell Content: |             |               |             |               |        |      |            |                 |        |
|--------------------------------------------------------------------------|-------------|---------------|-------------|---------------|--------|------|------------|-----------------|--------|
|                                                                          | employee_id | department_id | location_id | employee_name | gender | age  | hire_date  | designation     | salary |
| ▶                                                                        | 1           | 1             | 1           | Arun Kumar    | M      | 28   | 2025-01-01 | developer       | 45000  |
|                                                                          | 2           | 2             | 2           | Karthik       | M      | 35   | 2025-01-20 | Finance Officer | 55000  |
| *                                                                        | NULL        | NULL          | NULL        | NULL          | NULL   | NULL | NULL       | NULL            | NULL   |

- ✦ Limit the acceptable values for the "gender" field to only 'M' or 'F'.

```

173      /* Limit the acceptable values for the "gender" field to only 'M' or 'F' */
174
175      select * from employees
176      WHERE gender not in ('M','F');
177

```

- ✦ Enforce a condition to ensure that the employee's age is 18 or above.

```

186
187
188      select * from employees
189      where age >=18;

```

| Result Grid   Filter Rows:   Edit:   Export/Import:   Wrap Cell Content: |             |               |             |               |        |      |            |                 |        |
|--------------------------------------------------------------------------|-------------|---------------|-------------|---------------|--------|------|------------|-----------------|--------|
|                                                                          | employee_id | department_id | location_id | employee_name | gender | age  | hire_date  | designation     | salary |
| ▶                                                                        | 1           | 1             | 1           | Arun Kumar    | M      | 28   | 2025-01-01 | developer       | 45000  |
|                                                                          | 2           | 2             | 2           | Karthik       | M      | 35   | 2025-01-20 | Finance Officer | 55000  |
|                                                                          | 3           | 3             | 3           | Rahul         | M      | 18   | 2026-07-27 | Intern          | 10000  |
|                                                                          | 4           | 4             | 4           | Meena         | F      | 20   | 2026-07-27 | Software Tester | 15000  |
| *                                                                        | NULL        | NULL          | NULL        | NULL          | NULL   | NULL | NULL       | NULL            | NULL   |

employees 14 x

- ✦ Automatically assign the current date to the "hire\_date" field if not specified.

| Edit:   Export/Import:   Wrap Cell Cor |               |        |     |            |                 |
|----------------------------------------|---------------|--------|-----|------------|-----------------|
| location_id                            | employee_name | gender | age | hire_date  | designation     |
| 1                                      | Arun Kumar    | M      | 28  | 2025-01-01 | developer       |
| 2                                      | Karthik       | M      | 35  | 2025-01-20 | Finance Officer |
| 3                                      | Rahul         | M      | 18  | 2026-07-27 | Intern          |
| 4                                      | Meena         | F      | 20  | 2026-07-27 | Software Tester |

## Conclusion

- The Employees Database was successfully created using SQL DDL commands.
- Tables for Employees, Departments, and Locations were designed with proper structure.
- Constraints such as PRIMARY KEY, FOREIGN KEY, NOT NULL, CHECK, and DEFAULT were applied to ensure data integrity.
- The hire\_date field was set to automatically assign the current date if not specified.
- Sample data was inserted and tested against constraints to validate correctness.
- Invalid entries (like age below 18 or wrong gender values) were prevented by constraints.
- Relationships between tables were established using foreign keys, ensuring consistency.

Thank You

Entri Elevate